N.C. AUTONOMOUS COLLEGE, JAJPUR

Department of Computer Applications

A PROJECT REPORT ON

UPASTHITI

College Attendance & Academic Management System

(Attendance · Timetable · Results · Library · Study Notes)

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS (BCA)

SUBMITTED BY

Name of the Student	Roll No.
Bhagyashree Panigrahi	BCA A19
Dhurti Prava Das	BCA A62

Department of Computer Applications

N.C. Autonomous College, Jajpur

Academic Session: _____

CERTIFICATE

This is to certify that the project report entitled "Upasthiti: College Attendance & Academic Management System" is a bonafide work carried out by Bhagyashree Panigrahi (Roll No. BCA A19) and Dhurti Prava Das (Roll No. BCA A62), students of the Bachelor of Computer Applications (BCA) programme, Department of Computer Applications, N.C. Autonomous College, Jajpur.

The project is submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Computer Applications. To the best of my knowledge, the work has not been submitted elsewhere for the award of any other degree or diploma.

_____

Head of the Department

Department of Computer Applications

N.C. Autonomous College, Jajpur

DECLARATION

We hereby declare that the project report entitled "Upasthiti: College Attendance & Academic Management System", submitted to the Department of Computer Applications, N.C. Autonomous College, Jajpur, in partial fulfilment of the requirements for the award of the degree of Bachelor of Computer Applications (BCA), is a record of original work done by us.

The results embodied in this report have not been submitted to any other university or institution for the award of any degree or diploma.

Place: Jajpur

Date: _____

_____

Bhagyashree Panigrahi

Roll No. BCA A19

_____

Dhurti Prava Das

Roll No. BCA A62

ACKNOWLEDGEMENT

We express our sincere gratitude to the Principal of N.C. Autonomous College, Jajpur for providing us with the facilities and the opportunity to carry out this project.

We are deeply thankful to the Head of the Department of Computer Applications for constant encouragement and support throughout the course of this work.

We also thank all the faculty members and staff of the department, our friends and classmates who helped us test the application, and our families for their patience and support.

Bhagyashree Panigrahi
Roll No. BCA A19

Dhurti Prava Das
Roll No. BCA A62

ABSTRACT

Attendance in most colleges is still taken on paper registers and then copied into spreadsheets. This is slow, error-prone and easy to misuse through proxy attendance. Students find out about attendance shortages only near the examinations, parents are rarely informed, and timetables, marks, notices, library records and study material are spread across separate registers, notice boards and messaging groups.

Upasthiti (Sanskrit/Odia for "presence") is a web-based College Attendance & Academic Management System built for the BCA and BBA departments of N.C. Autonomous College, Jajpur. Students mark their campus presence from their phones with a location-verified (geofenced) punch-in and punch-out, and teachers record period-wise class attendance against the timetable. The system calculates attendance percentages automatically, warns students who fall below 75 % and alerts parents by email when attendance becomes critical.

Beyond attendance, the application manages the academic calendar, timetables with clash detection, internal assessment marks with a verification workflow, SGPA/CGPA and report cards, a digital notice board, monthly parent reports, a library management module (catalogue, issue, return, renewals and fines) and a study-notes module where teachers upload notes and students search and download them.

The system has seven roles (Admin, HOD, Teacher, Class Coordinator, Student, Parent and Librarian), each with its own portal. It is built with Next.js 16, React 19, TypeScript, Tailwind CSS and the Prisma ORM over an SQLite database (PostgreSQL in production). The user interface follows a Sambalpuri-inspired visual theme that reflects the culture of Odisha. The application was verified with 363 automated end-to-end checks and 11 unit tests, all of which pass.

Keywords: *attendance management, geofencing, Next.js, Prisma, academic management, library management, web application*

TABLE OF CONTENTS

CERTIFICATE 1
DECLARATION 2
ACKNOWLEDGEMENT 3
ABSTRACT 4
CHAPTER 1: INTRODUCTION 7
1.1 Background 7
1.2 Problem Statement 7
1.3 Objectives 7
1.4 Scope of the Project 8
1.5 Why This Project Was Chosen 8
1.6 Organisation of the Report 8
CHAPTER 2: SYSTEM ANALYSIS 9
2.1 Existing System 9
2.2 Proposed System 9
2.3 Feasibility Study 10
CHAPTER 3: REQUIREMENTS SPECIFICATION 11
3.1 Functional Requirements 11
3.2 Non-Functional Requirements 11
3.3 Hardware Requirements 12
3.4 Software Requirements 12
CHAPTER 4: SYSTEM DESIGN 14
4.1 System Architecture 14
4.2 User Roles and Portals 15
4.3 Module Design 16
4.4 Attendance Percentage Calculation 17
4.5 Geofenced Punch Process 17
4.6 Database Design 18
4.7 Data Flow (Level 0) 22
4.8 Marks and Results Workflow 23
4.9 Project Structure 23
CHAPTER 5: TECHNOLOGY USED 25
5.1 Next.js 25
5.2 React 25
5.3 TypeScript 25
5.4 Tailwind CSS 25

5.5 Prisma ORM and the Database 25
5.6 Supporting Libraries 25
5.7 Development Tools 26
5.8 Why This Technology Stack Was Chosen 26
CHAPTER 6: IMPLEMENTATION AND SCREENSHOTS 28
6.1 Public Pages 28
6.2 Admin Portal 29
6.3 Student Portal 32
6.4 Teacher Portal 36
6.5 HOD Portal 38
6.6 Library Portal 39
6.7 Parent Portal 39
6.8 Responsive Design and Mobile Use 40
CHAPTER 7: TESTING 41
7.1 Testing Strategy 41
7.2 Test Results 41
7.3 Sample Test Cases 41
7.4 Tools Used for Testing and Debugging 42
7.5 Problems Faced and How They Were Solved 43
CHAPTER 8: SECURITY AND PRIVACY 44
CHAPTER 9: CONCLUSION AND FUTURE SCOPE 45
9.1 Conclusion 45
9.2 Limitations 45
9.3 Future Scope 45
REFERENCES 46
APPENDIX: SOURCE CODE AND DEMO ACCESS 47
A.1 Source Code Repository 47
A.2 How to Run the Project 47
A.3 Demonstration Accounts 47

CHAPTER 1: INTRODUCTION

1.1 Background

Regular attendance is one of the most important factors in a student's academic performance, and colleges are required to maintain a minimum attendance (usually 75 %) for a student to be eligible for the semester examinations. In most colleges, including ours, attendance is taken by calling out roll numbers and marking a paper register. At the end of the month the register is totalled by hand, and shortages are announced only a few weeks before the examinations.

Information technology now makes it possible to take attendance digitally, compute percentages instantly, and share the information with students, teachers and parents in real time. Almost every student owns a smartphone with GPS, which allows attendance to be tied to physical presence on the campus.

1.2 Problem Statement

- Manual registers take 5–10 minutes of every class and the totals contain calculation errors.
- Proxy attendance (one student answering for another) is hard to detect.
- Students and parents have no way to see the current attendance percentage until it is too late.
- Timetables, marks, notices, library records and study notes are kept in separate registers, notice boards and WhatsApp groups, so information is scattered and often lost.
- Library issue and return registers make it difficult to track overdue books and fines.
- Teachers share notes through messaging groups, which makes old material hard to find.

1.3 Objectives

- To build a secure web application where students mark campus attendance with a location-verified punch-in/punch-out from their phones.
- To let teachers mark period-wise class attendance quickly against the timetable.
- To calculate attendance percentages automatically and warn students, mentors and parents at the 75 % and 30 % thresholds.

- To manage the academic calendar, timetables, marks, results and notices in one place.
- To provide a library management module and a searchable study-notes module for the BCA and BBA departments.
- To give each user role (admin, HOD, teacher, student, parent, librarian) a simple dashboard of its own.

1.4 Scope of the Project

The project covers the two self-financed departments of the college: Bachelor of Computer Applications (BCA) and Bachelor of Business Administration (BBA), each with three years and one or more sections. The application runs in any modern web browser on a desktop or mobile phone and can be installed on a phone's home screen as a Progressive Web App. It can be extended to other departments simply by adding them from the admin portal.

The scope was kept realistic for a three-year BCA project. The project does not include a native mobile app, online fee payment or hostel management, and real SMS delivery needs a paid gateway; these are listed as future scope in Chapter 9.

1.5 Why This Project Was Chosen

Attendance is a problem that every student and teacher of the college meets every day. Registers are totalled by hand, shortages are discovered only near the examinations, and information about timetables, marks, notices, library books and notes is spread over many places. A project that solves this is useful in real life, can be tested with real users, and uses the main topics of the BCA course: web development, database design, security and software testing.

1.6 Organisation of the Report

Chapter 2 analyses the existing system and the feasibility of the proposed system. Chapter 3 lists the requirements. Chapter 4 describes the system design and database. Chapter 5 introduces the technologies used. Chapter 6 presents the implemented modules with screenshots. Chapter 7 describes testing, Chapter 8 security, and Chapter 9 concludes the report with future scope.

CHAPTER 2: SYSTEM ANALYSIS

2.1 Existing System

In the existing system, attendance is marked on paper registers maintained by each subject teacher. The class coordinator collects the registers at the end of the month and calculates percentages by hand. Notices are pinned on a physical notice board or forwarded on WhatsApp. Marks are entered into spreadsheets, and the library keeps a handwritten issue register.

Limitations of the existing system

- Time consuming and labour intensive.
- Prone to calculation errors and proxy attendance.
- No real-time visibility for students or parents.
- Data is duplicated across registers and spreadsheets.
- Difficult to generate reports such as defaulter lists.
- No central record of notices, notes or library fines.

2.2 Proposed System

Upasthiti replaces the registers with a single web application backed by a central database. Every action (a punch, an attendance mark, a mark entry, a book issue) is stored once and immediately reflected in every report and dashboard.

Advantages of the proposed system

- **Accurate:** percentages are calculated by the system using a single, documented formula.
- **Proxy-resistant:** the campus punch is accepted only within 100 m of the campus point and only from the student's approved phone.
- **Transparent:** students and parents can see attendance, timetable and results at any time.
- **Proactive:** automatic warnings below 75 % and critical alerts below 30 %, with monthly parent reports.
- **Integrated:** attendance, timetable, marks, notices, library and notes share the same data.
- **Auditable:** every important change is written to an audit log.

2.3 Feasibility Study

Technical feasibility

The project uses free, open-source and widely supported technologies (Node.js, Next.js, React, Prisma, SQLite/PostgreSQL). It runs on an ordinary computer during development and on any low-cost cloud server or college server in production. Users need only a web browser, so no special hardware is required.

Economic feasibility

All software used is free. The only running costs are a small server or hosting plan and, optionally, an SMS gateway. The time saved for teachers (several minutes per class) and office staff (monthly calculations) far outweighs these costs.

Operational feasibility

The user interface is simple, mobile-friendly and uses familiar terms (punch in, timetable, results). Each role sees only what it needs. Bulk import from Excel/CSV lets the office set up hundreds of students in minutes, so adoption is easy.

Schedule feasibility

The development was planned in ten phases (days), each delivering a working, tested set of features, followed by the library and study-notes modules. The plan was completed on schedule.

CHAPTER 3: REQUIREMENTS SPECIFICATION

3.1 Functional Requirements

Table 1: Functional requirements by module

Module	Main functions
Authentication	Sign-in with college ID and password, lockout after 5 failed attempts, one-time code on first sign-in and for password reset, forced password change, optional two-step sign-in for staff, privacy consent
Administration	Academic years, departments, sections, lab batches, subjects, teacher-subject mapping, users and roles, CSV/Excel bulk import
Campus attendance	Geofenced punch-in/out, GPS accuracy check, one approved device per student, offline queue, suspicious-movement flags, auto punch-out
Leave & regularisation	Leave requests with documents, regularisation of missed punches, approval by coordinator or HOD
Calendar	Holidays, exams, working Saturdays and events with notifications
Timetable	Period timings, rooms, weekly grid editor, clash detection, versions, cancel/substitute/extra classes, printable view
Class attendance	Period-wise P / L / A / Leave / Medical marking, 24-hour edit window with HOD approval, percentage engine, risk lists
Marks & results	Assessment scheme, grid or Excel entry, Draft → Submitted → Verified → Locked → Published workflow, grades, SGPA/CGPA, report card, analytics
Parent communication	Monthly report emails, 75 % warning and 30 % critical alerts, email queue with retries and delivery log
Notices	Audience targeting, urgent/pinned notices, attachments, scheduling, expiry, read statistics, public notice board
Library	Book catalogue and search, issue, return, renewal, borrowing limit, overdue fines, due-date reminders
Study notes	Teachers upload PDF/Word/PowerPoint/image notes per subject; students search and open them; HOD moderation
Reports & audit	Dashboards, defaulter lists, Excel/CSV exports, audit log viewer

3.2 Non-Functional Requirements

- **Security:** hashed passwords, signed session cookies, role-based access on every page and action, rate limiting, security headers.
- **Privacy:** consent before collecting location data (guardian consent for students under 18); location is checked only at the moment of punching.

- **Usability:** responsive design for phones and desktops, simple language, accessible forms.
- **Performance:** pages are pre-rendered where possible and stream personalised parts; database queries are indexed.
- **Reliability:** emails are queued and retried; offline punches are stored on the phone and sent later.
- **Maintainability:** TypeScript throughout, validated inputs, automated tests, database migrations.
- **Portability:** runs on Windows or Linux; SQLite for development, PostgreSQL for production.

3.3 Hardware Requirements

Table 2: Hardware requirements

Component	Server / development	Client (user)
Processor	Dual-core 2 GHz or better	Any smartphone or PC
Memory (RAM)	4 GB minimum, 8 GB recommended	2 GB
Storage	10 GB free	–
Other	Internet connection	GPS (for punching), internet connection

3.4 Software Requirements

Table 3: Software requirements

Item	Used in this project
Operating system	Windows 10/11 (development), Linux (production)
Runtime	Node.js 20 or newer (developed on Node.js 24)
Framework	Next.js 16.4 (App Router) with React 19.3
Language	TypeScript 5
Styling	Tailwind CSS 4
Database & ORM	SQLite (development), PostgreSQL (production), Prisma ORM 6.19
Libraries	jose (JWT sessions), bcryptjs (password hashing), zod (validation), nodemailer (email), fflate (Excel files)

Item	Used in this project
Tools	Visual Studio Code, Git, Google Chrome, Prisma Studio
Client	Any modern browser: Chrome, Edge, Firefox or Safari

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

Upasthiti follows a three-tier architecture built on the Next.js App Router:

- **Presentation tier:** React components rendered on the server and sent to the browser as HTML, with small interactive parts (forms, the punch screen, search boxes) running in the browser.
- **Application tier:** Next.js server components and server actions. Every page and every action first checks the user's session and role in a single data-access layer, validates the input with zod, performs the business logic and writes an audit log entry.
- **Data tier:** a relational database accessed through the Prisma ORM. Uploaded files (leave documents, notice attachments, notes) are stored in a private folder and served only after a permission check.

Background jobs (auto punch-out, monthly reports, the email queue, library reminders and scheduled publishing) run through a protected job endpoint that a scheduler calls every few minutes. Figure 1 shows how the parts fit together and how data moves between them: a request from the browser is checked by the route guard, then by the data-access layer (session and role), validated by the server action, processed by the business logic and finally stored through Prisma.

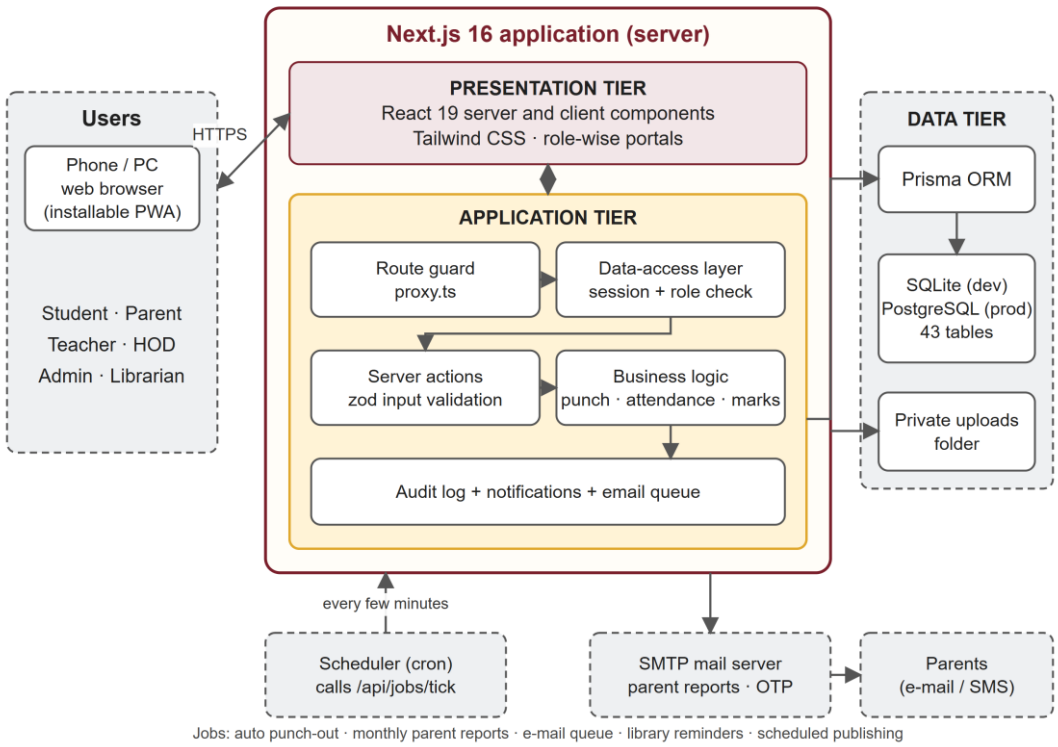


Figure 1: System architecture of Upasthiti

4.2 User Roles and Portals

Table 4: Roles and what each one can do

Role	Portal	Key responsibilities
Admin (office)	Admin	Set up the academic structure and users, geofence and college hours, calendar, timetables, results publishing, parent reports, library, audit log
HOD	HOD	Department overview, approve leave and attendance edits, verify marks, manage department timetable, notices and notes
Teacher	Teacher	Mark class attendance, enter marks, upload study notes, post notices, view own timetable
Class coordinator (mentor)	Teacher	Watch mentees' attendance risk, approve leave and regularisation requests
Student	Student	Punch in/out, view attendance, timetable, calendar and results, apply for leave, search library and notes
Parent	Parent	View child's attendance, calendar and results; receive monthly reports and alerts
Librarian	Library	Manage the catalogue, issue and return books, renewals and fines

A single person can hold more than one role (for example, a teacher who is also the HOD). Such users choose a portal after signing in and can switch between portals at any time.

The use case diagram in Figure 2 shows what each kind of user can do with the system.

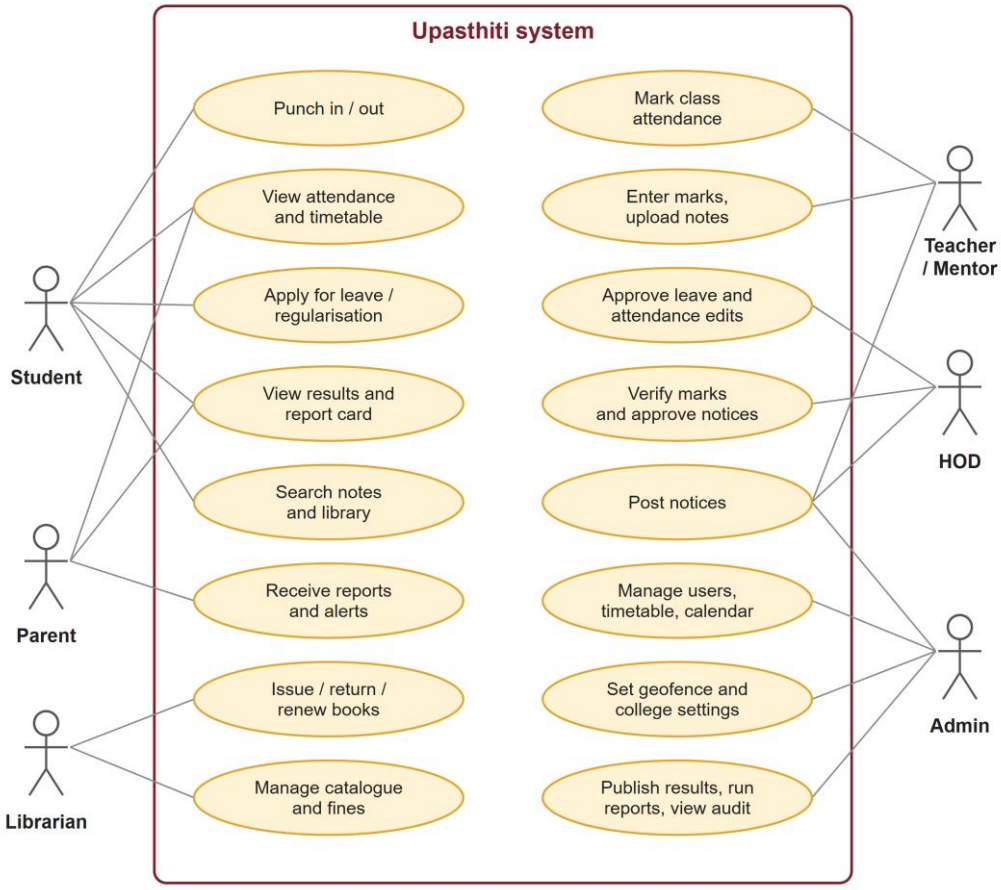


Figure 2: Use case diagram

4.3 Module Design

The application is divided into the following modules, each implemented as a set of pages, server actions and library functions:

- Authentication and account security
- Administration and bulk import
- Geofenced campus attendance
- Leave and regularisation
- Academic calendar
- Timetable management

- Class attendance and the percentage engine
- Marks, results and report cards
- Parent communication and email queue
- Notices and public notice board
- Library management
- Study notes
- Dashboards, analytics, exports and audit log

4.4 Attendance Percentage Calculation

Attendance is calculated per subject and overall using the formula:

Attendance % = (Classes attended + Approved-leave classes*) ÷ Classes held × 100

Here, "classes held" are only those periods in which attendance was actually taken, so holidays, exam days and cancelled periods never count against a student. "Late" is counted as attended. *Whether approved leave counts as present or is excluded altogether is a setting chosen by the college. Optionally, a Present mark counts only if the student also punched in on campus that day.

Table 5: Attendance status levels

Attendance	Status	Action taken by the system
75 % and above	Safe (green)	Shown on dashboards
30 % to below 75 %	Warning (amber)	Student and mentor are warned; parent report shows warning; system shows how many classes are needed to reach 75 %
Below 30 %	Critical (red)	Immediate alert email (and SMS if configured) to parents with a copy to the class coordinator

4.5 Geofenced Punch Process

- The student opens the punch screen; the browser asks for the phone's location.
- The location, its accuracy and the device identifier are sent to the server.
- The server checks that the device is the student's approved phone, the GPS accuracy is within 30 m, the day is a working day and the time is within college hours.

- The server calculates the distance from each campus point using the Haversine formula and accepts the punch only if the student is within 100 m.
- Unusual cases (clock differences, impossible movement speed) are flagged for the admin. If the phone is offline, the punch is queued and sent later.

Figure 3 shows the order of the checks. A punch is rejected as soon as one check fails, and every punch, accepted or rejected, is stored together with the reason so that the office can review it.

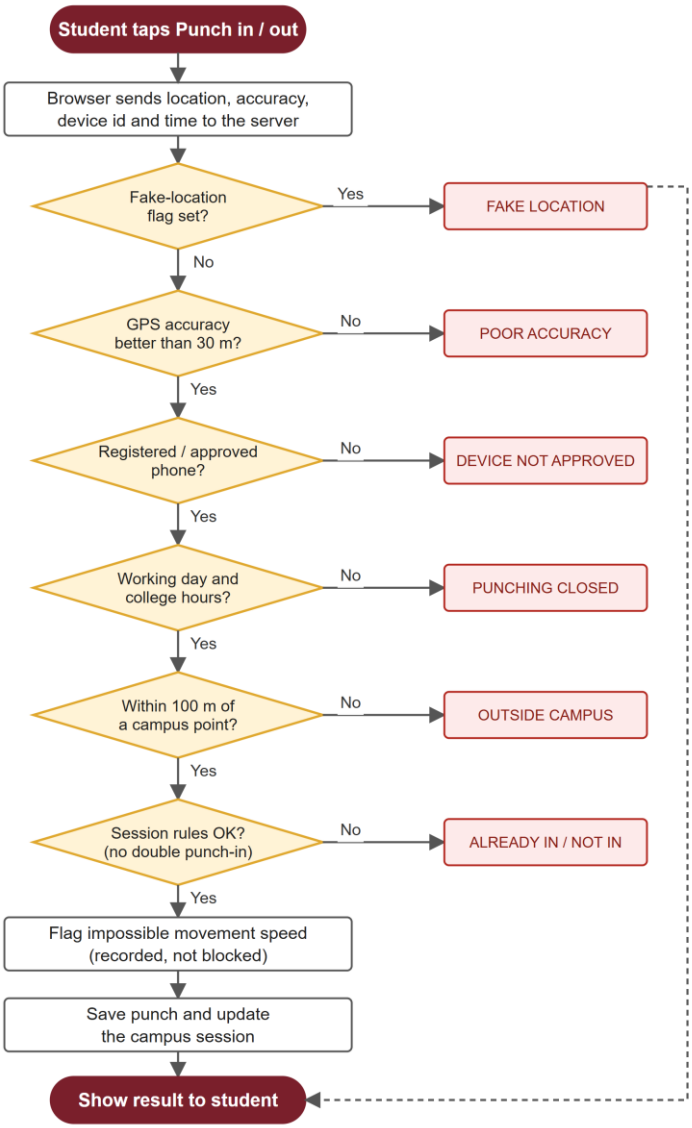


Figure 3: Flowchart of the geofenced punch process

4.6 Database Design

The database is relational and contains 43 tables, managed through Prisma migrations. The entity-relationship diagrams in Figure 4 and Figure 5 show the most important tables and how they are connected; Table 6 describes them in words.

How the tables are connected

- **Users and profiles:** every person who can sign in has one row in User. A student, a teacher or a parent has a profile row (Student, Teacher, Parent) linked to the User. A student can have many parents and a parent many students, so they are joined through the ParentStudent table.
- **Academic structure:** a Department has many Sections and Subjects. A Section has many Students. TeacherSubject says which teacher teaches which subject to which section.
- **Class attendance:** each period that was held is one ClassSession (a section, a subject, a teacher and a date). It has one AttendanceRecord for every student, so a student appears in many records and a session has many records.
- **Campus attendance:** a Student has many CampusSessions (one per campus day), and every PunchRecord belongs to a student, usually to a CampusSession and to the Geofence that accepted it.
- **Marks:** an Assessment belongs to a subject and a section and has one MarkEntry per student. Every change made after locking is saved in MarkRevision.
- **Library and notes:** a Book has many BookLoans and each BookLoan belongs to one student. A Note belongs to a subject and a department.

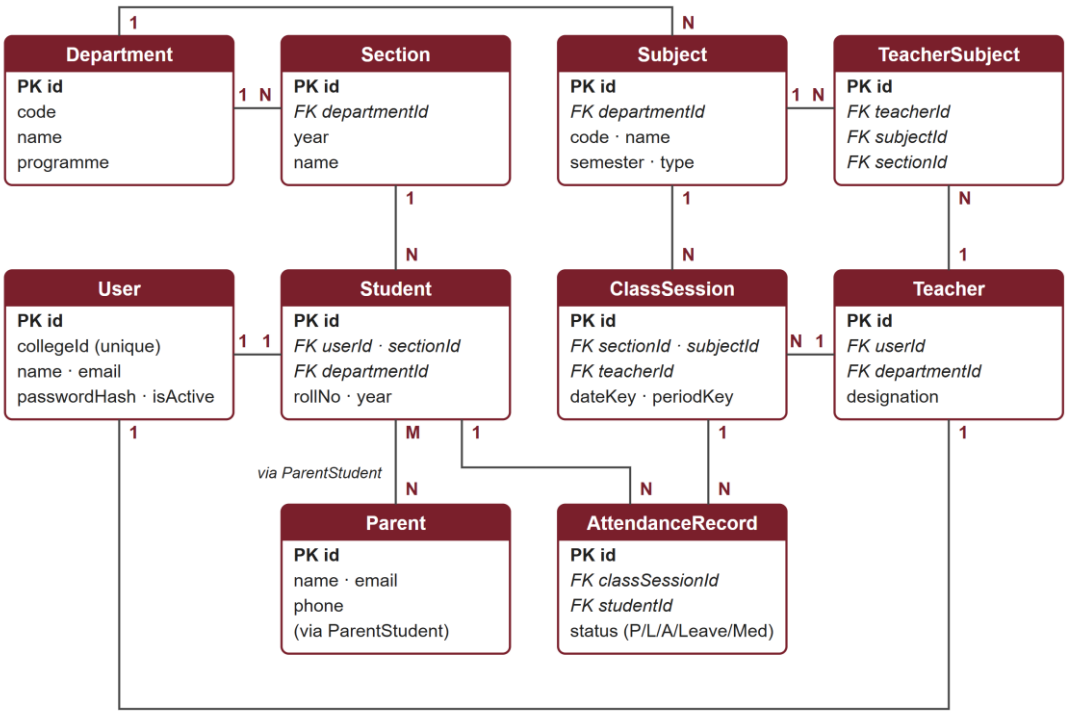


Figure 4: ER diagram: users, academic structure and class attendance

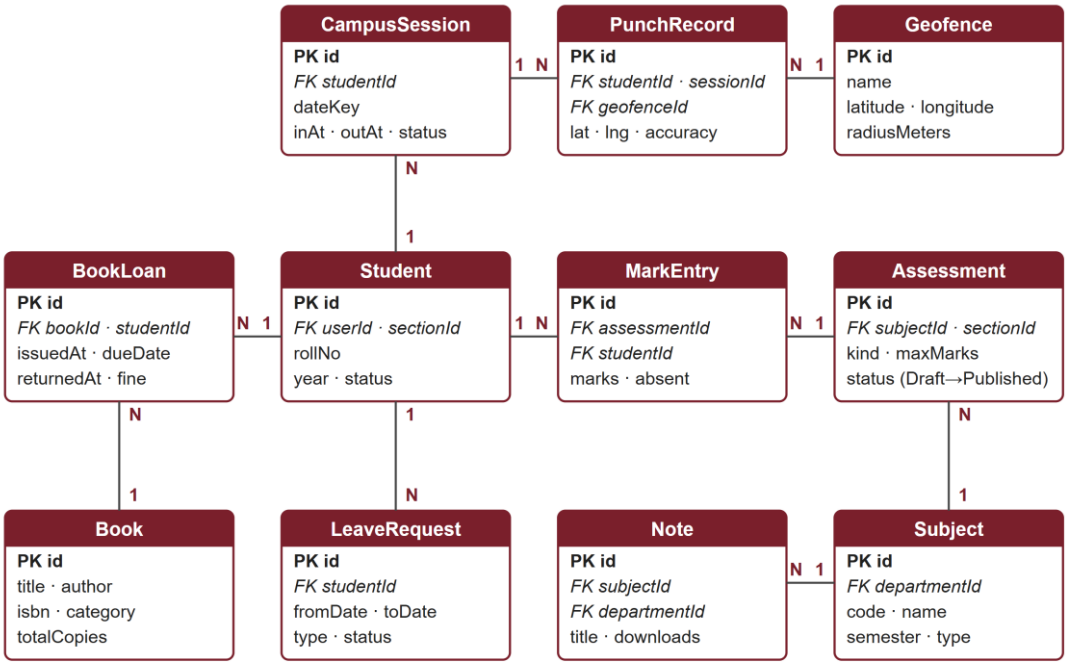


Figure 5: ER diagram: campus attendance, leave, marks, library and notes

Normalisation

The tables are in third normal form. Each fact is stored once: a student's department is reached through the section, subject names are stored only in Subject, and attendance

percentages are never stored but calculated from the records, so a correction in one place is immediately correct everywhere. Many-to-many relationships (parents and students, teachers and subjects) use their own tables, and the periods of a timetable are stored as rows instead of repeating columns. Status values such as the attendance status or the marks status are database enums, which prevents invalid values.

Table 6: Main database tables

Table	Purpose	Important fields
User / UserRole	Every person who can sign in, and their roles	collegeId, name, email, passwordHash, isActive; role, departmentId, sectionId, subjectId
Department / Section / Batch	Academic structure (BCA, BBA; years; sections; lab batches)	code, name, programme; year, name
Student / Teacher / Parent	Profiles linked to users	rollNo, sectionId, year, status; designation; name, email (ParentStudent: relationship)
Subject / TeacherSubject	Subjects and who teaches them in which section	code, name, semester, type; teacherId, subjectId, sectionId
Geofence / Device / PunchRecord	Campus points, approved phones and every punch	latitude, longitude, radiusMeters; deviceHash, status; accuracyMeters, distanceMeters, flags
CampusSession	One campus day per student (punch-in to punch-out)	dateKey, inAt, outAt, status
Timetable / TimetableSlot / PeriodChange	Weekly timetables and one-off changes	effectiveFrom; dayOfWeek, periodNumber, subjectId, teacherId, roomId; type
ClassSession / AttendanceRecord	Each period held and each student's mark	dateKey, periodKey, sectionId, subjectId; status (present, absent, late, on leave, medical)
LeaveRequest / RegularisationRequest	Requests and their approval	fromDate, toDate, reason, status, decidedById
Assessment / MarkEntry / MarkRevision	Exams, marks and every change to them	kind, maxMarks, status, publishedAt; marks, absent; oldMarks, newMarks, reason
Notice / NoticeRead / NoticeHistory	Notices, read receipts and edit history	title, audience, urgent, pinned, publishAt, expiresAt

Table	Purpose	Important fields
Book / BookLoan	Library catalogue and loans	title, author, isbn, totalCopies; issuedAt, dueDate, returnedAt, fine
Note	Uploaded study notes	title, subjectId, sectionId, tags, filePath, downloads
EmailLog / ReportRun	Email queue and monthly report runs	toAddress, subject, status, attempts; monthKey, status
AuditLog / Setting	Audit trail and college-wide settings	actorId, action, entityType, entityId, details; key, value

4.7 Data Flow (Level 0)

At the highest level, the system receives inputs from six external entities and produces outputs for them:

- **Students → system:** punches, leave requests, searches; system → students: attendance, timetable, results, notes, notices.
- **Teachers → system:** class attendance, marks, notes, notices; system → teachers: timetables, class lists, analytics.
- **HOD → system:** approvals and verifications; system → HOD: department reports.
- **Admin → system:** academic setup, users, settings, calendar; system → admin: dashboards, exports, audit log.
- **Librarian → system:** books, issues and returns; system → librarian: loans, overdue list, fines.
- **System → parents:** monthly reports and attendance alerts by email/SMS, and the parent portal.

Figure 6 shows the same flows as a Level 0 data flow diagram, with the database as the single data store.

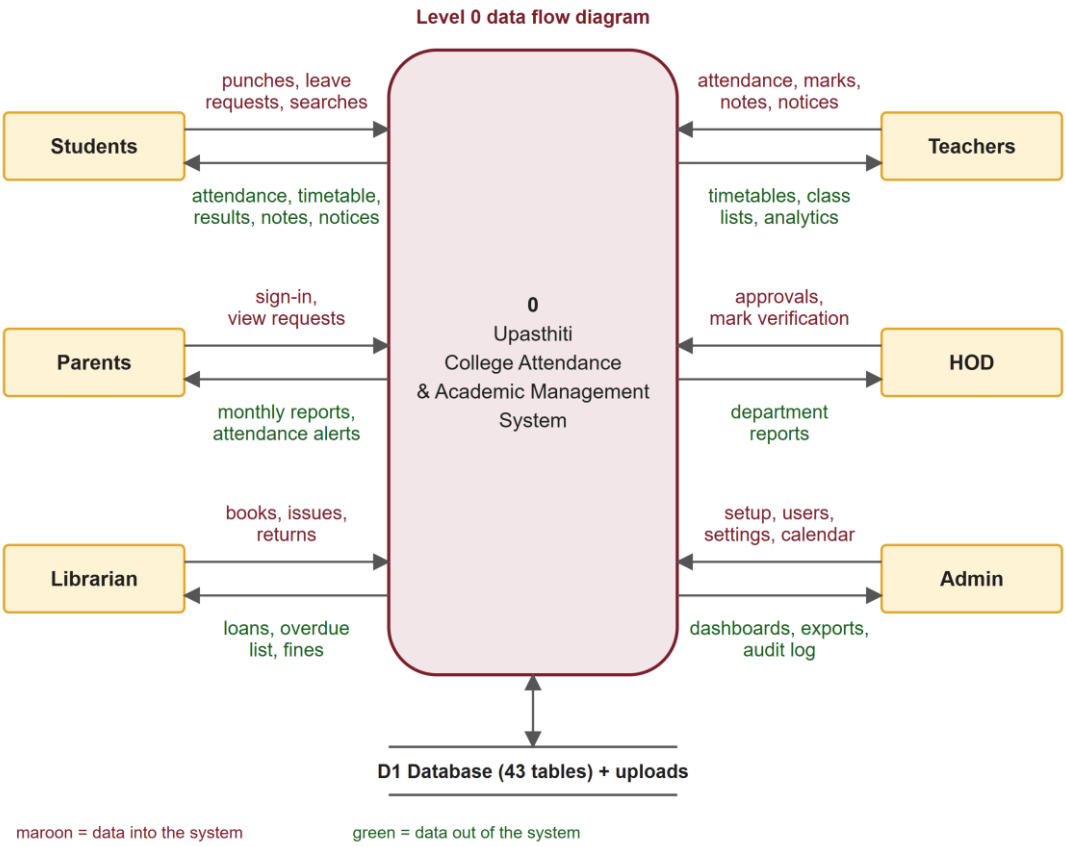


Figure 6: Level 0 data flow diagram

4.8 Marks and Results Workflow

Marks pass through five states so that a wrong mark cannot reach students by mistake. The teacher enters the marks (Draft) and submits them. The HOD verifies them or returns them to the teacher with a note. Only the admin can lock verified marks and publish them, and every later change to locked marks is written to a revision history with the reason.

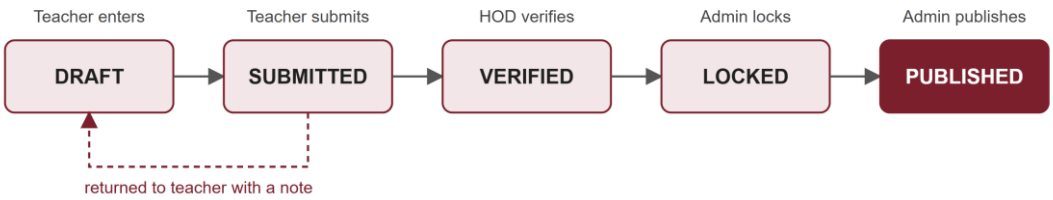


Figure 7: Marks and results workflow

4.9 Project Structure

The source code is organised by purpose so that every feature has an obvious place:

Table 7: Folders of the project

Folder	Contents
src/app	78 pages, one folder per portal (admin, hod, teacher, student, parent, library), the server actions of each feature, account and sign-in pages, and the API route handlers (punch, jobs, exports, file downloads)
src/components	28 reusable interface components: portal shell, forms, attendance sheet, marks grid, timetable editor and views, calendar, charts, notes browser
src/lib	41 modules of business logic that contain no screen code: punch rules, attendance percentage, results, library, notes, notices, reports, email queue, permissions, sessions, validation helpers
src/proxy.ts	First route filter that sends visitors without a session to the sign-in page
prisma	schema.prisma (43 models), migrations, the seed script with demonstration data and the admin-creation script
tests	Unit tests and end-to-end test scripts

Business rules live in src/lib so that the same function is used by pages, actions and tests, and shared checks such as role verification and input validation are written once and reused instead of being repeated in every page.

CHAPTER 5: TECHNOLOGY USED

5.1 Next.js

Next.js is an open-source React framework developed by Vercel for building full-stack web applications. Version 16 with the App Router is used. It provides file-based routing, server components (which render on the server and send HTML to the browser), server actions (functions that run on the server when a form is submitted), streaming with Suspense, and a proxy layer for request filtering. Using Next.js allowed both the user interface and the server logic to be written in one project and one language.

5.2 React

React is a JavaScript library for building user interfaces from reusable components. React 19 is used, including the useActionState hook for forms that show validation errors and success messages returned by the server.

5.3 TypeScript

TypeScript is a typed superset of JavaScript. Static types catch many mistakes at compile time, make the code self-documenting and allow the database types generated by Prisma to be used safely throughout the application.

5.4 Tailwind CSS

Tailwind CSS is a utility-first CSS framework. It was used to build a responsive layout and a custom Sambalpuri-inspired theme with maroon, turmeric (haldi) and cream colours, an ikat pattern band and temple-border motifs.

5.5 Prisma ORM and the Database

Prisma is an Object-Relational Mapper for Node.js. The database schema is written in a single schema file; Prisma generates type-safe query functions and migration files from it. SQLite is used during development because it needs no installation, and the same schema works with PostgreSQL for production.

5.6 Supporting Libraries

- **jose:** creates and verifies signed JSON Web Tokens stored in an HTTP-only session cookie.
- **bcryptjs:** hashes passwords with a salt so that they are never stored in plain text.

- **zod:** validates every form input on the server before it reaches the database.
- **nodemailer:** sends emails through an SMTP server (in development, emails are saved to an outbox folder).
- **fflate:** reads Excel (.xlsx) files for bulk import of students and marks.

5.7 Development Tools

- Visual Studio Code: code editor
- Git: version control
- Google Chrome DevTools: debugging and testing on mobile screen sizes
- Prisma Studio: browsing the database
- ESLint: code-quality checks

5.8 Why This Technology Stack Was Chosen

The project needs a website that works on a phone and a computer, a secure sign-in for seven kinds of users, a reliable relational database and scheduled background work. The stack below was chosen because one language and one project can cover all three layers.

Table 8: Technology stack and reasons for choosing it

Layer	Technology	Why it was chosen
Frontend	React 19, Tailwind CSS 4	Reusable components for the many similar screens (tables, forms, dashboards); Tailwind gives a responsive layout and the custom Sambalpuri theme without writing separate stylesheets; both run in every modern browser, so students need to install nothing.
Backend	Next.js 16 (Node.js) server components, server actions and route handlers	Frontend and backend live in one project, so there is no separate API server to deploy or keep in sync. Server actions let forms call server code directly and keep all permission checks on the server.
Language	TypeScript	The same language on both sides, and the compiler catches wrong field names and types before the code runs.
Database	SQLite (development), PostgreSQL (production) through Prisma ORM	Attendance, marks and library data are highly related, so a relational database with foreign keys is the natural fit. SQLite needs no installation for development and the same schema runs on PostgreSQL in production.

Layer	Technology	Why it was chosen
Security	jose, bcryptjs, zod	Signed session cookies, salted password hashing and server-side input validation are small, well-known libraries and avoid writing security code by hand.
Email	nodemailer	Sends parent reports and one-time codes through any SMTP server.
Testing	Node.js test runner, TypeScript compiler, ESLint	No extra framework is needed; the same tools run on any computer that has Node.js.

CHAPTER 6: IMPLEMENTATION AND SCREENSHOTS

The complete application has 78 pages and about 17,000 lines of TypeScript code in 202 source files. This chapter presents the most important screens. All screenshots are taken from the running application with the demonstration data supplied with the project.

6.1 Public Pages

Home page

The home page introduces Upasthiti with the college name, the main features and the latest public notices. The Sambalpuri ikat band and the chakra emblem form the visual identity of the application.

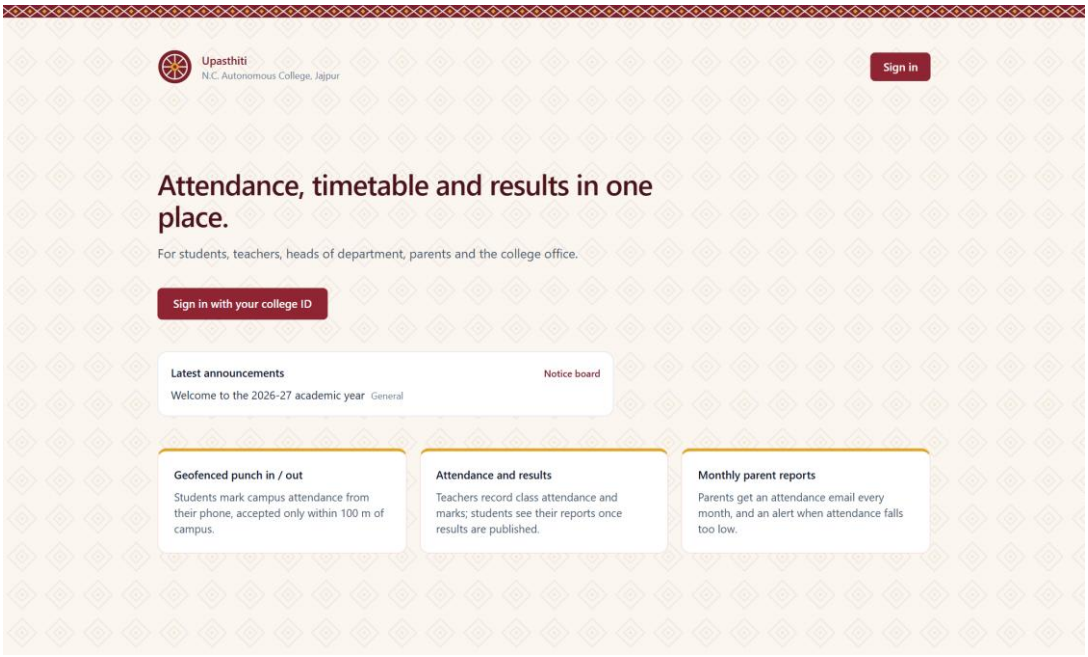


Figure 8: Home page

Sign-in page

Users sign in with their college ID and password. After five wrong attempts the account is temporarily locked. First-time users receive a one-time code by email or SMS and must set a new password.

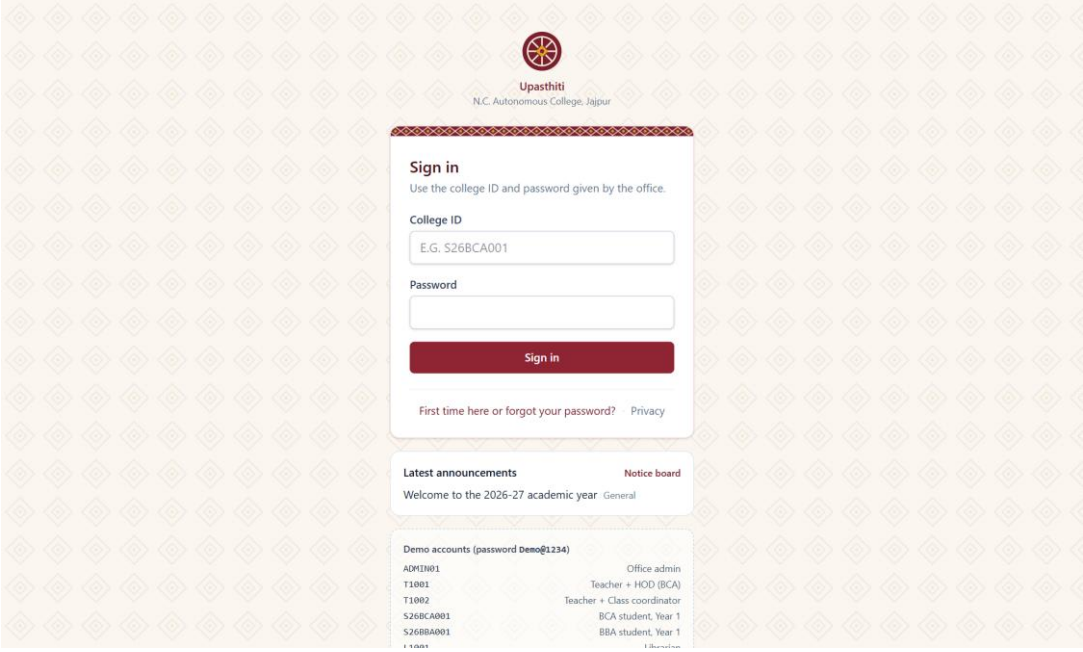


Figure 9: Sign-in page

Public notice board

Notices marked as public can be read without signing in, with search, categories and an archive.

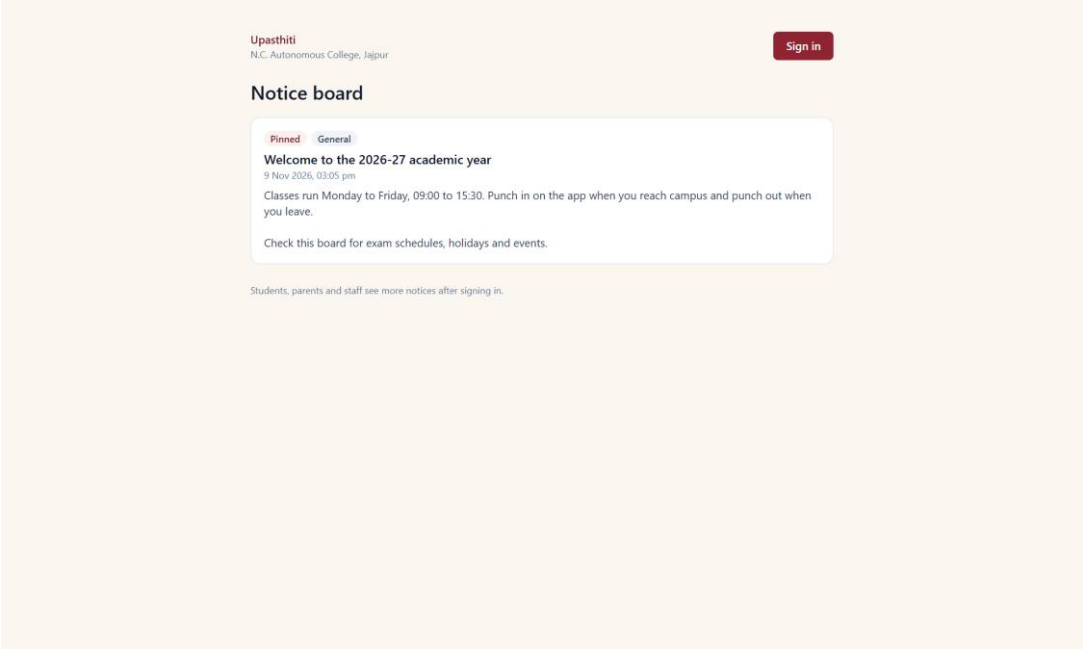


Figure 10: Public notice board

6.2 Admin Portal

Admin dashboard

The admin dashboard shows today's campus attendance, punch exceptions, pending tasks and shortcuts to every setup screen.

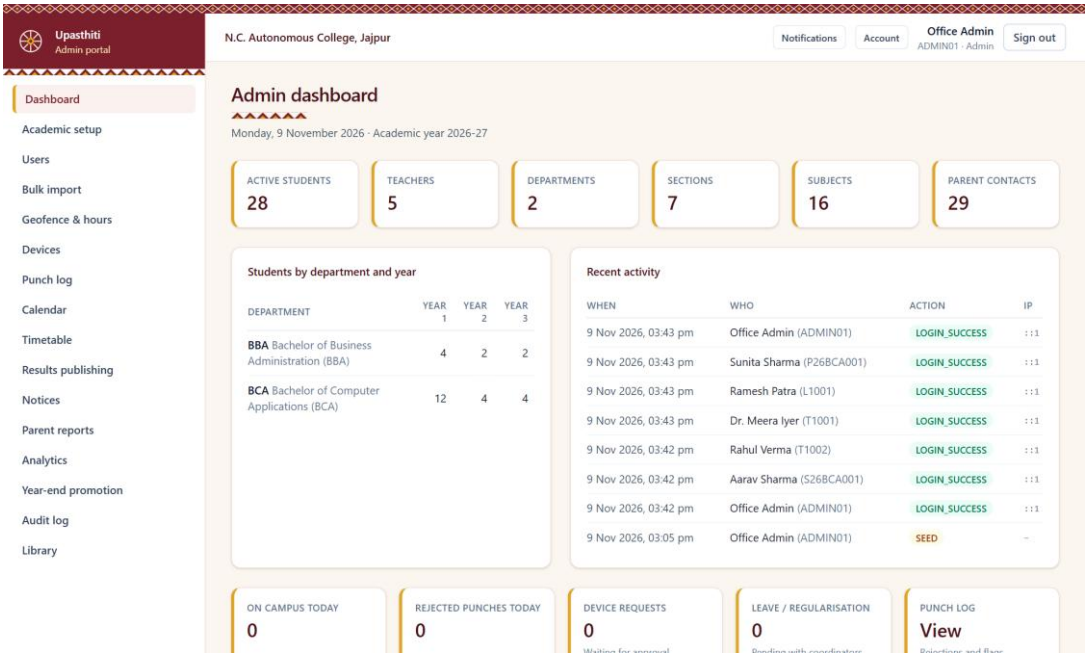


Figure 11: Admin dashboard

User management

The admin creates users individually or imports them from Excel/CSV with a preview, assigns roles and departments, and resets passwords.

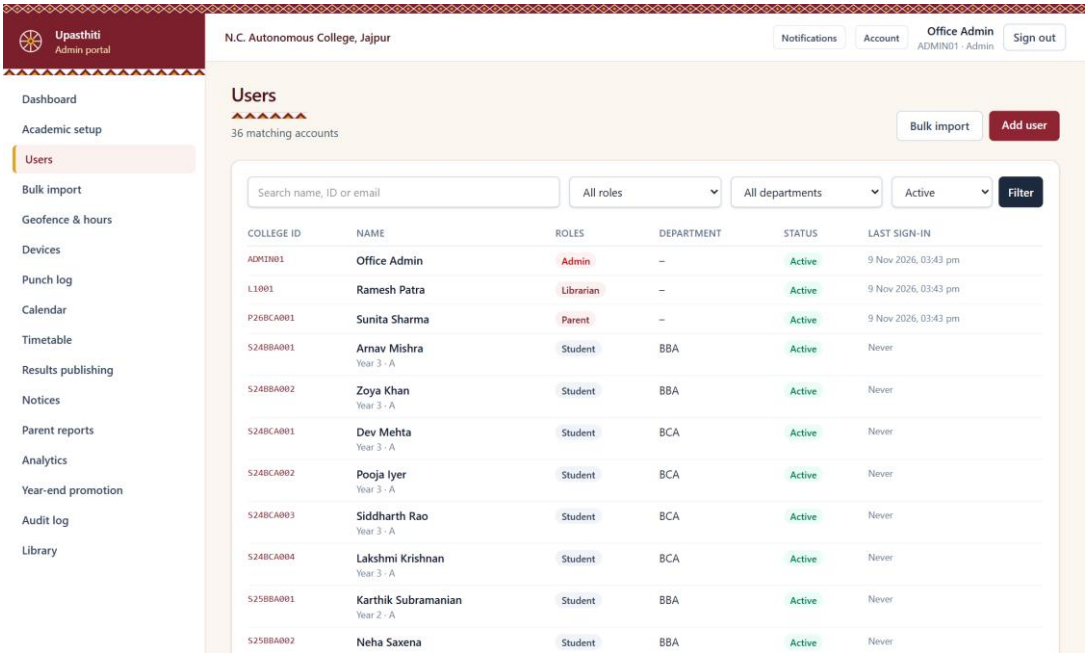


Figure 12: User management

Timetable editor

Timetables are edited on a weekly grid. The system detects clashes between teachers, rooms and sections before saving and keeps versions by effective date.

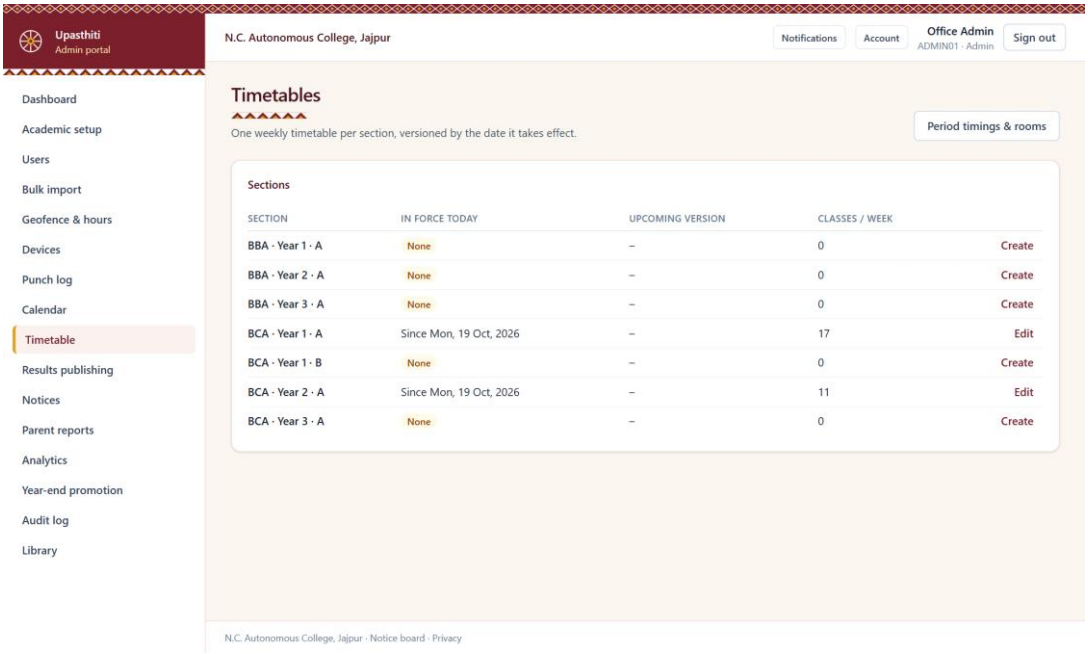


Figure 13: Timetable editor

Attendance analytics

Day-wise attendance charts, department and year comparisons and the defaulters list, all exportable to Excel.

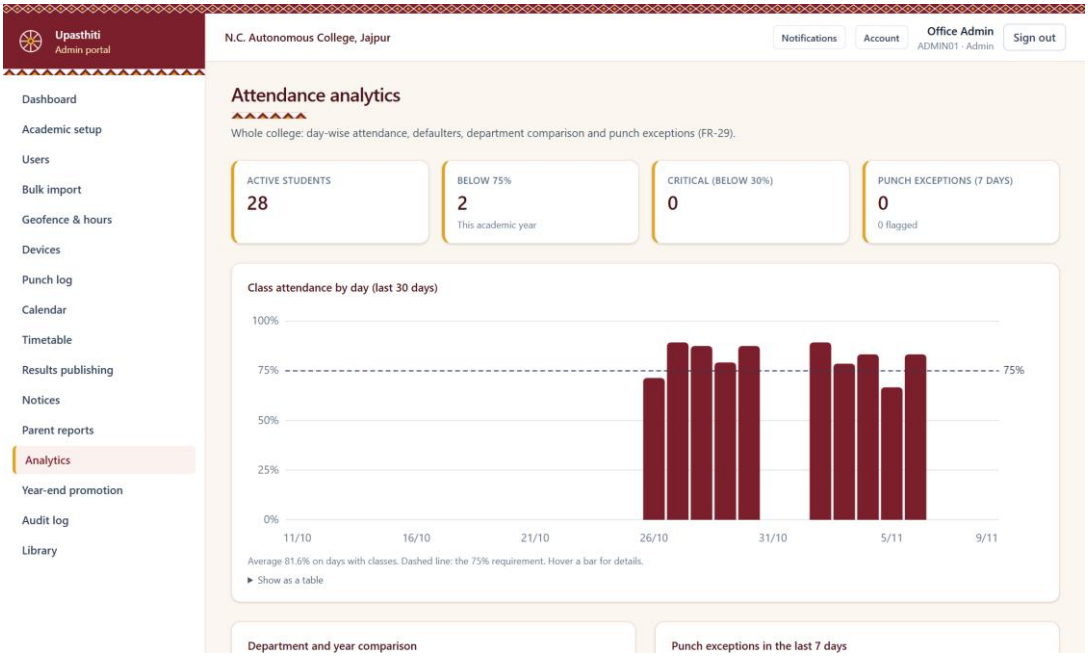


Figure 14: Attendance analytics

Parent reports

Monthly parent reports are scheduled on a chosen day. The admin can preview, pause, resend or trigger them manually and see the delivery log.

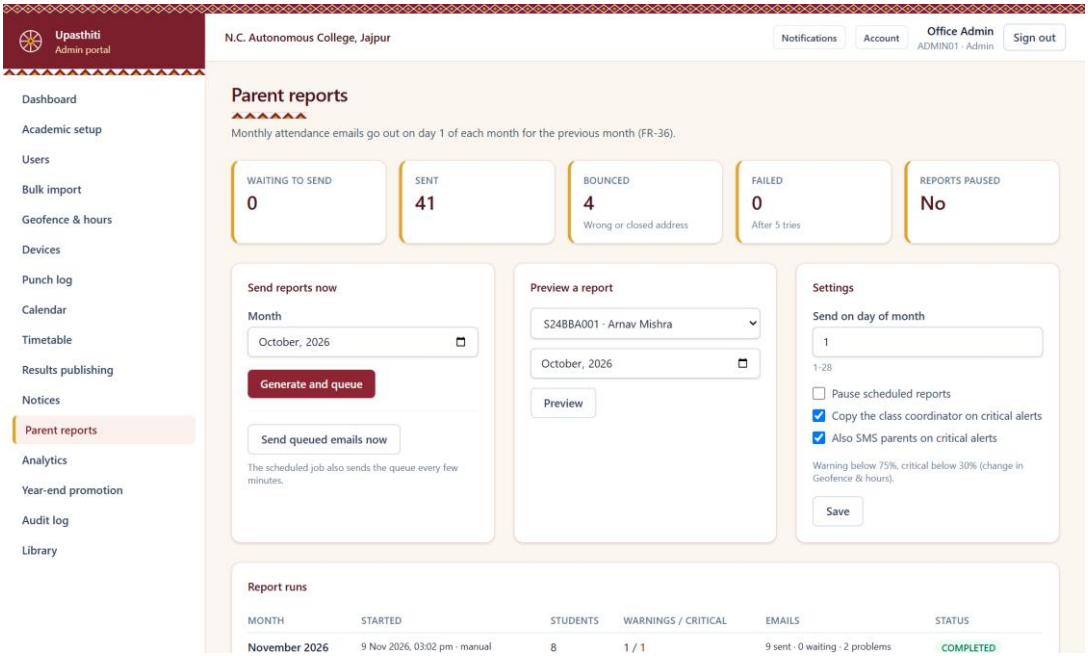


Figure 15: Parent reports

6.3 Student Portal

Student dashboard

The student sees today's campus status, profile, subjects with teachers and today's classes.

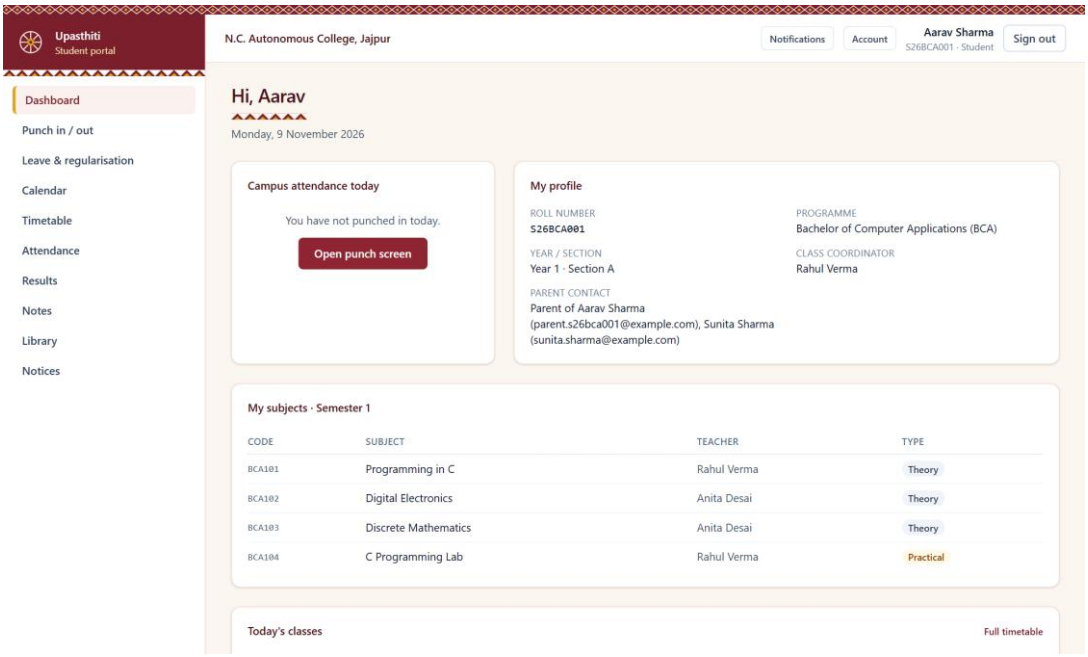


Figure 16: Student dashboard

Punch in / out

The punch screen reads the phone's location and sends it to the server, which verifies the distance from the campus, GPS accuracy, device and college hours before accepting the punch.

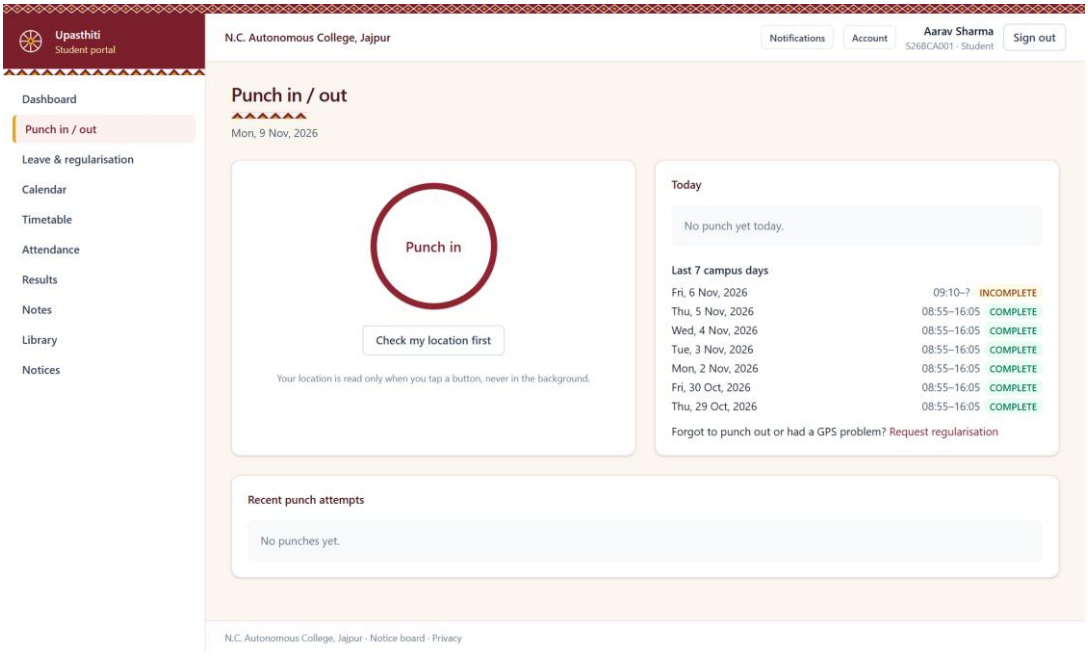


Figure 17: Punch in / out

My attendance

Subject-wise and overall attendance percentages with colour-coded status and the number of classes needed to reach 75 %.

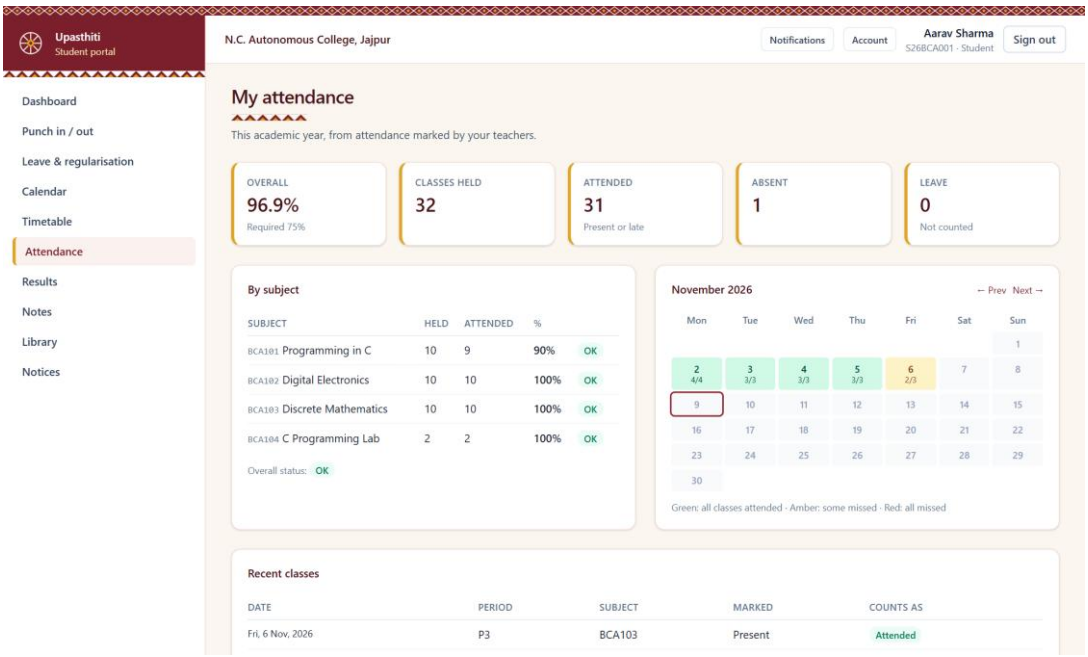


Figure 18: My attendance

Student timetable

The weekly timetable with rooms and teachers, including cancelled, substituted and extra classes.

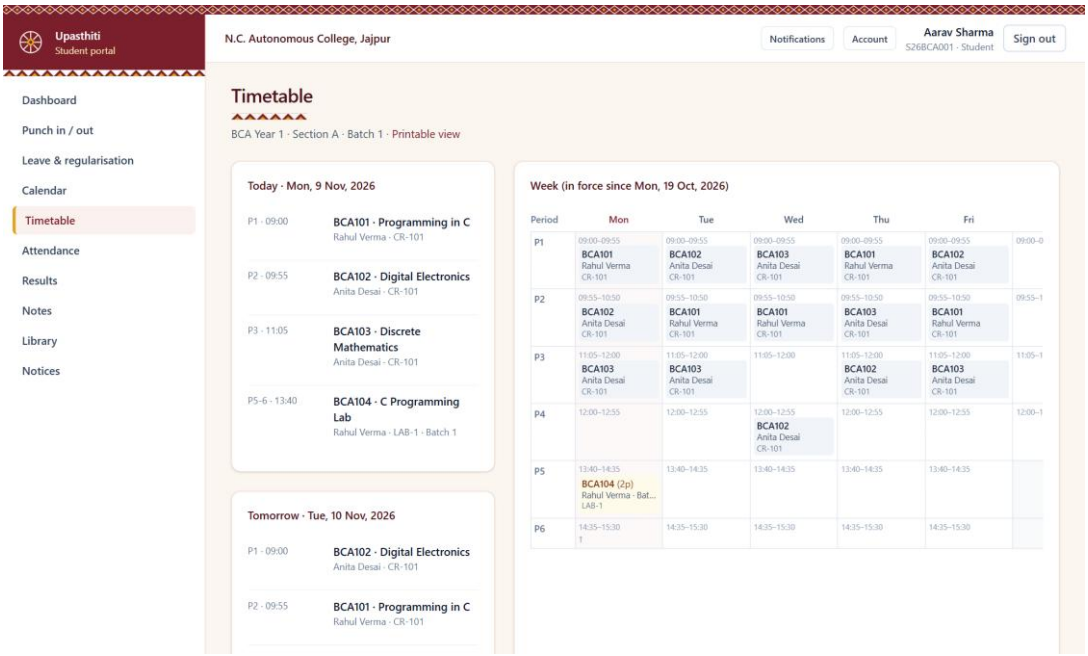


Figure 19: Student timetable

Results

Published marks, grades, SGPA/CGPA and a printable report card.

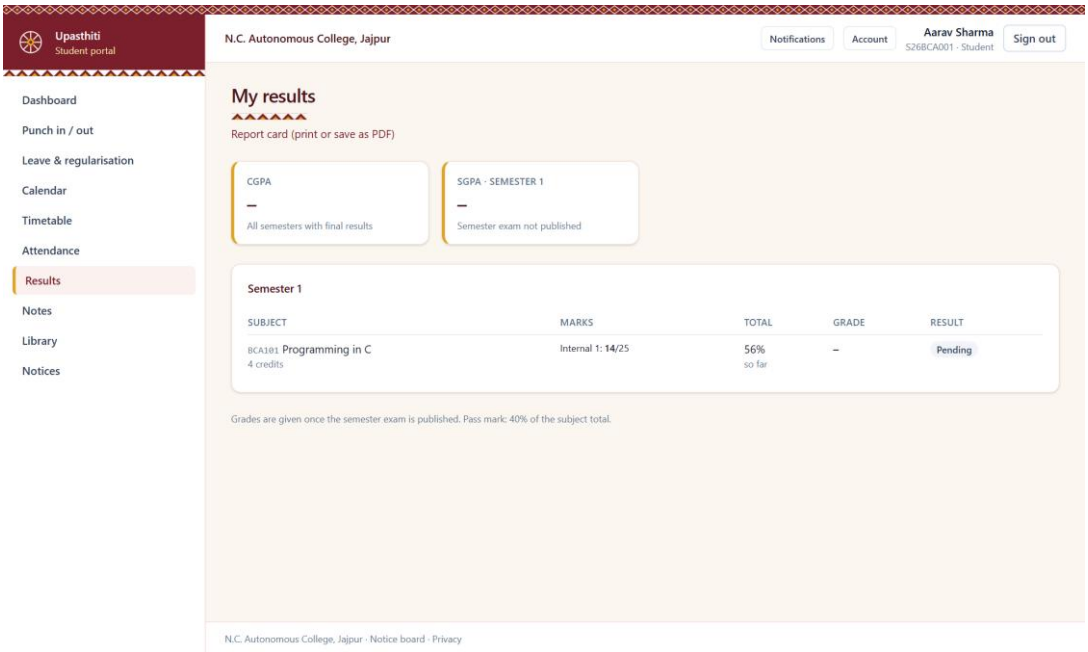


Figure 20: Results

Library for students

Students search the catalogue by title, author or ISBN, see availability and track their borrowed books, due dates and fines.

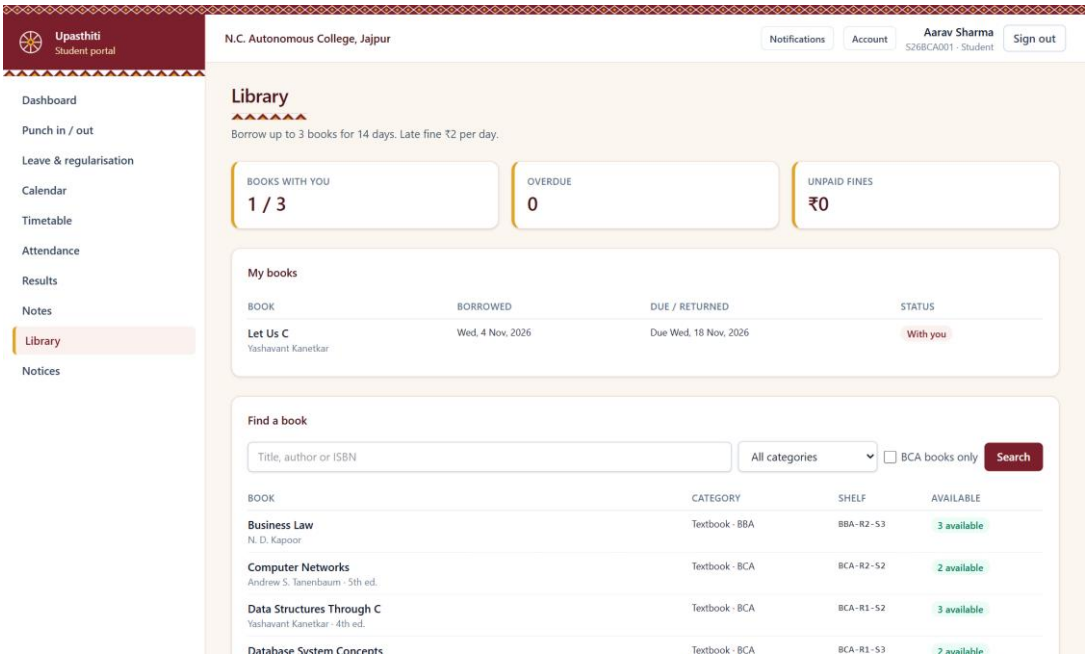


Figure 21: Library for students

Study notes for students

Students search notes by topic or subject and open them. Only notes for their own department, year and section are shown.

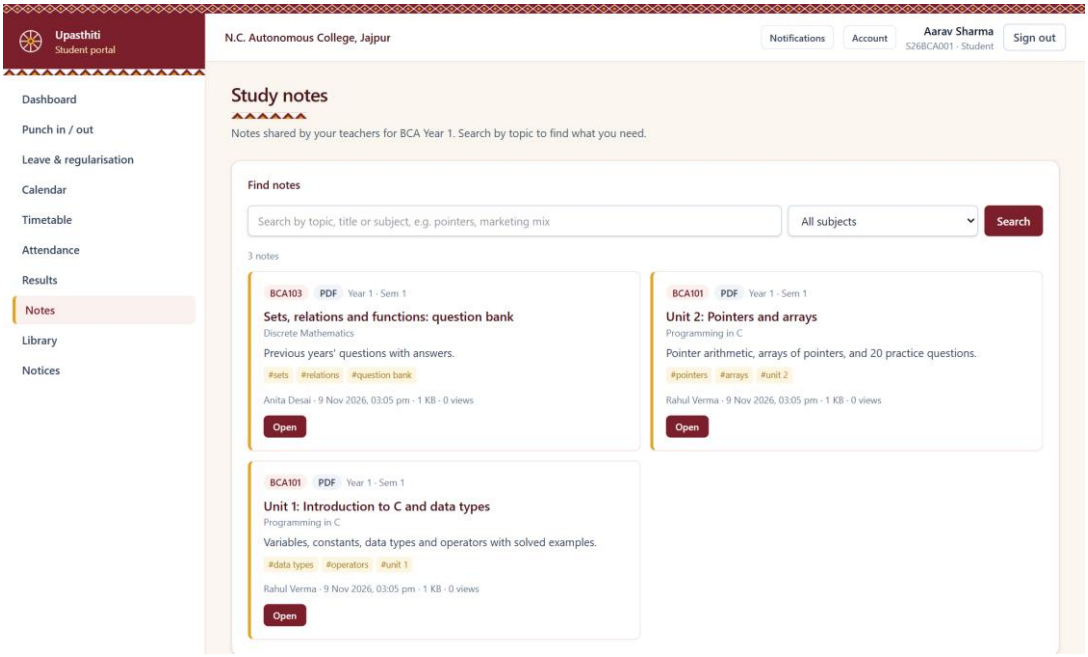


Figure 22: Study notes for students

6.4 Teacher Portal

Teacher dashboard

Today's classes, pending attendance, mentees at risk and approvals waiting for the teacher.

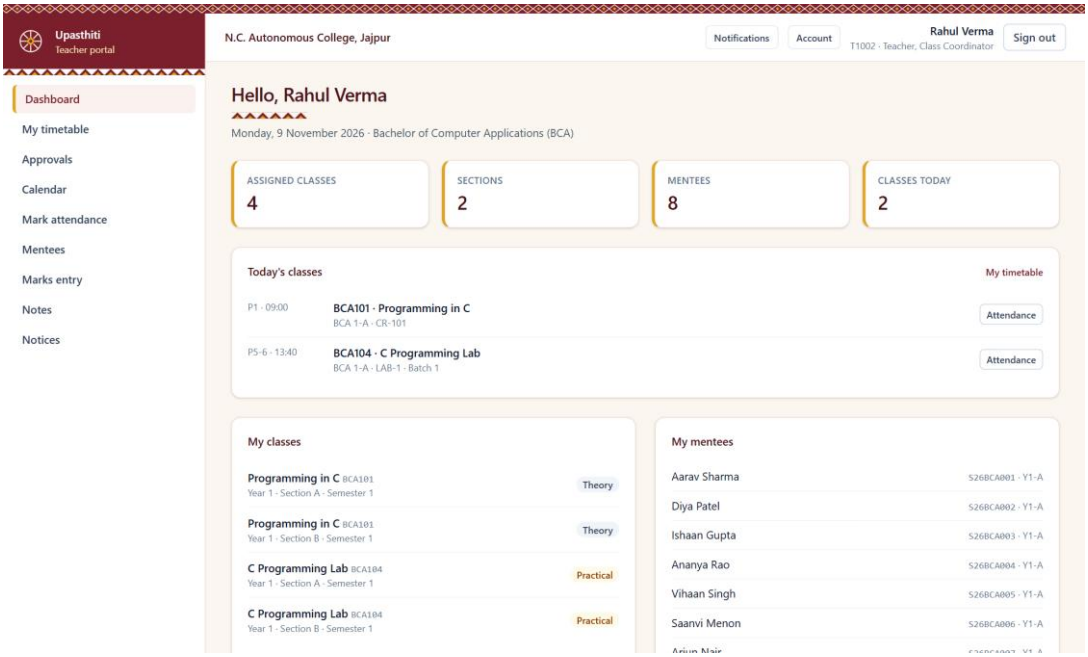


Figure 23: Teacher dashboard

Marking class attendance

For each timetabled period the teacher marks students Present, Late, Absent, Leave or Medical. Edits after 24 hours require HOD approval.

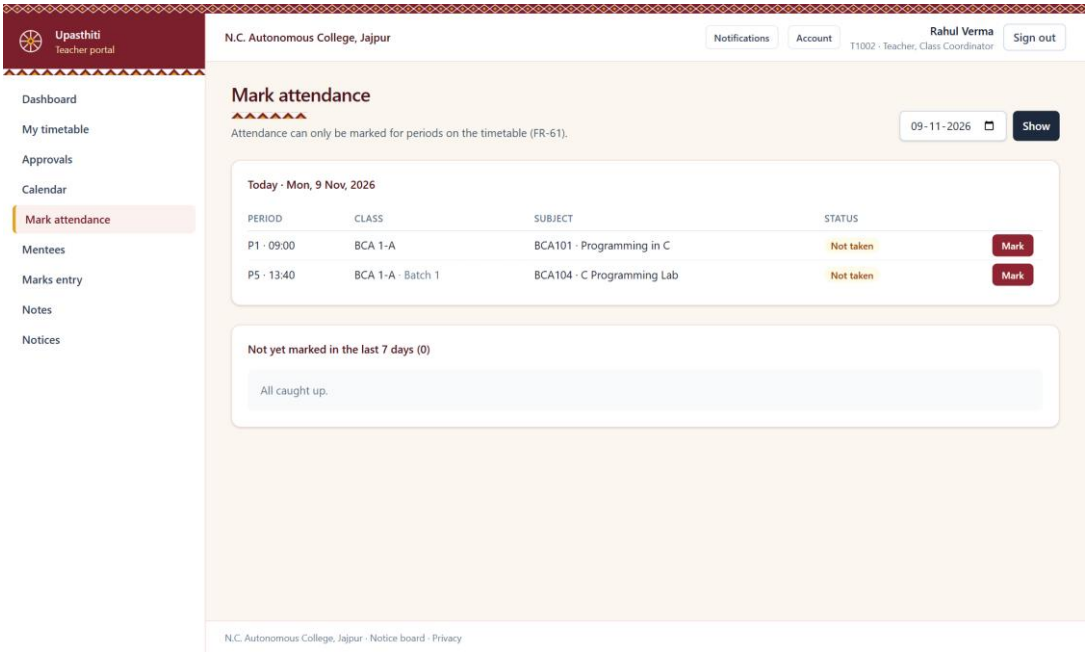


Figure 24: Marking class attendance

Marks entry

Marks are entered in a grid or uploaded from Excel and then submitted for HOD verification.

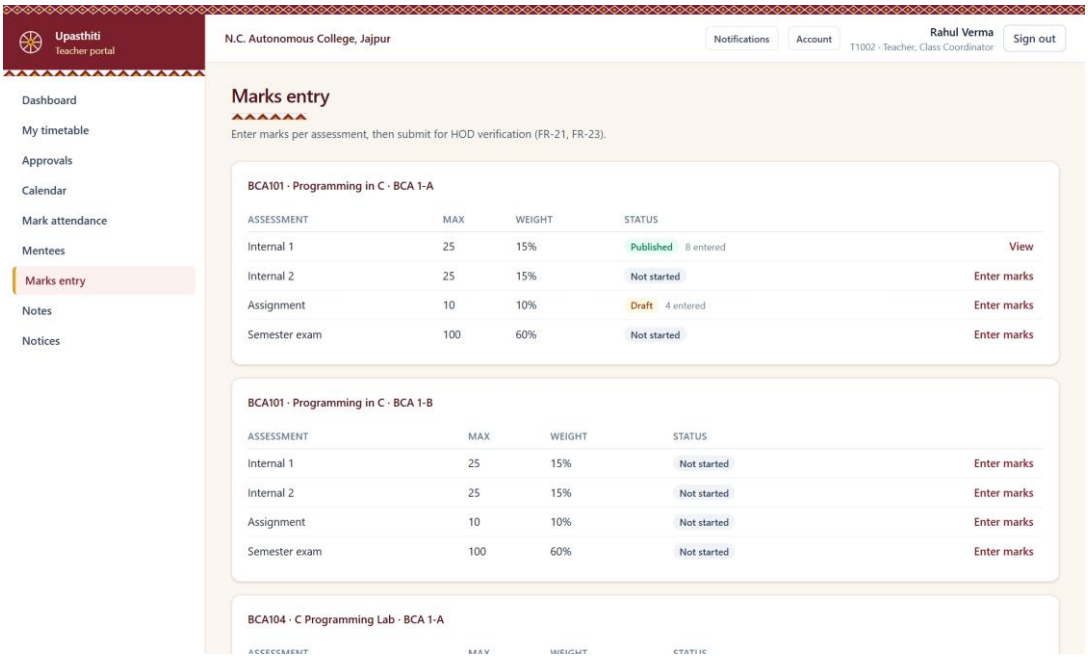


Figure 25: Marks entry

Uploading study notes

Teachers upload notes, slides or question banks for a subject and a section. Students are notified automatically.

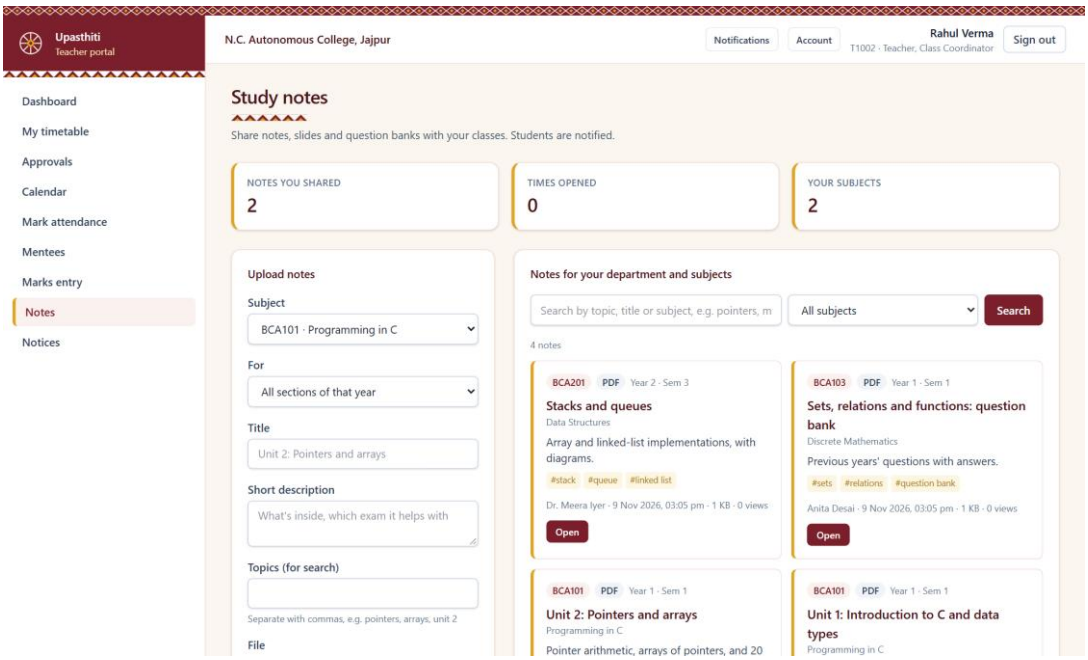


Figure 26: Uploading study notes

6.5 HOD Portal

HOD dashboard

The department overview shows student and teacher counts, and items waiting for the HOD such as marks verification and attendance edit requests.

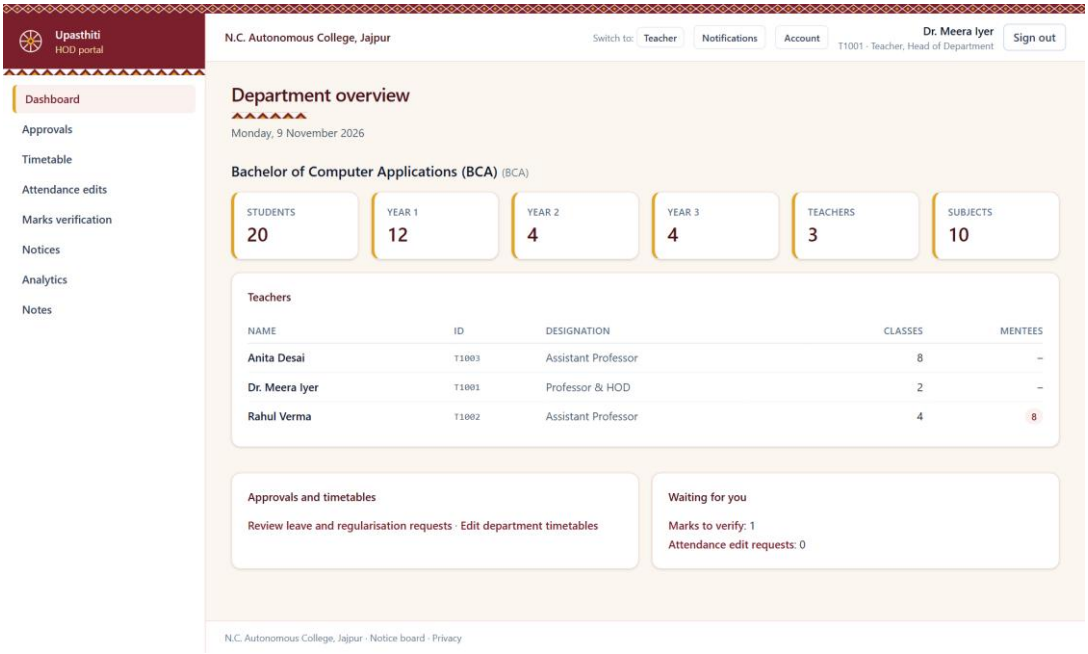


Figure 27: HOD dashboard

Department analytics

Attendance trends and defaulters for the HOD's department only.

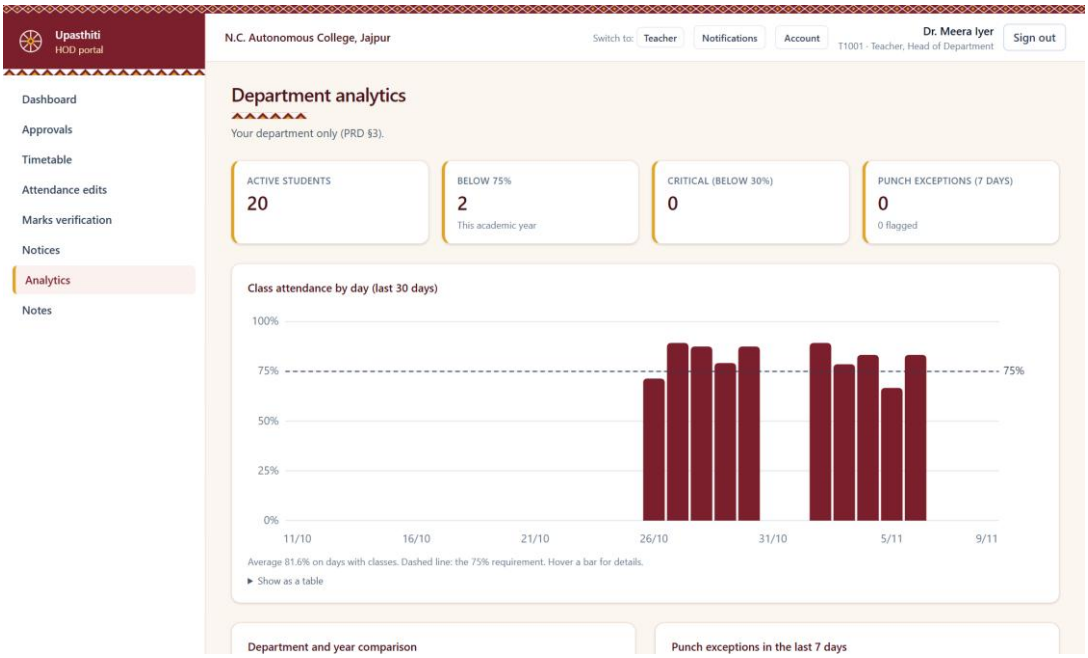


Figure 28: Department analytics

6.6 Library Portal

Library management

The librarian manages the catalogue and issues, returns and renews books. The system enforces the three-book limit and calculates overdue fines at ₹2 per day.

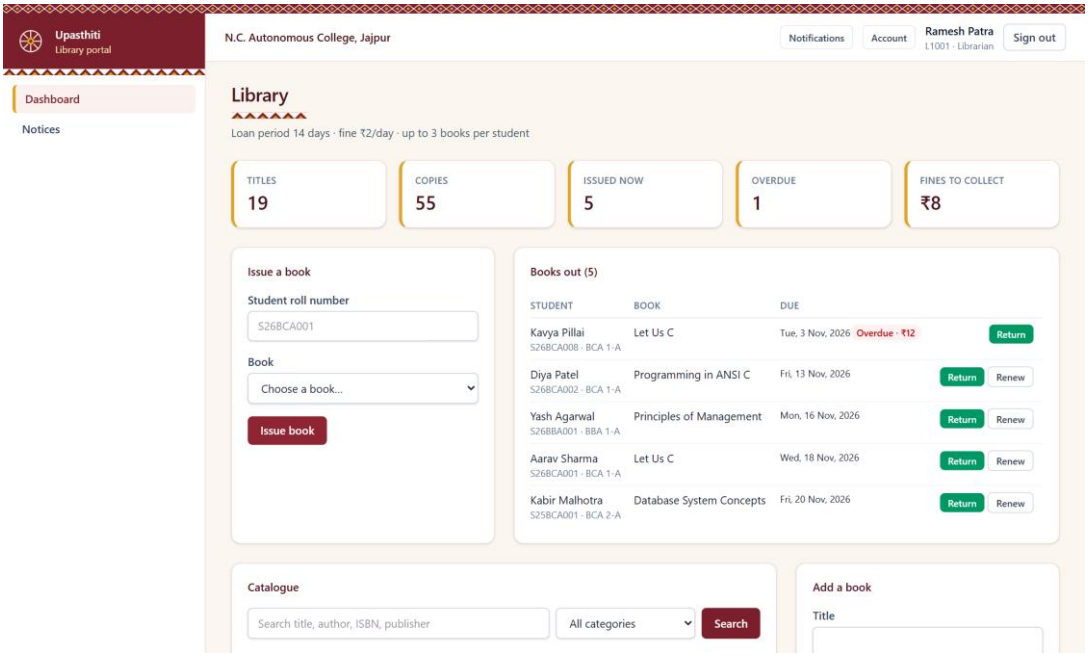


Figure 29: Library management

6.7 Parent Portal

Parent dashboard

Parents see each child's attendance, upcoming calendar events and published results. One parent account can be linked to more than one child.

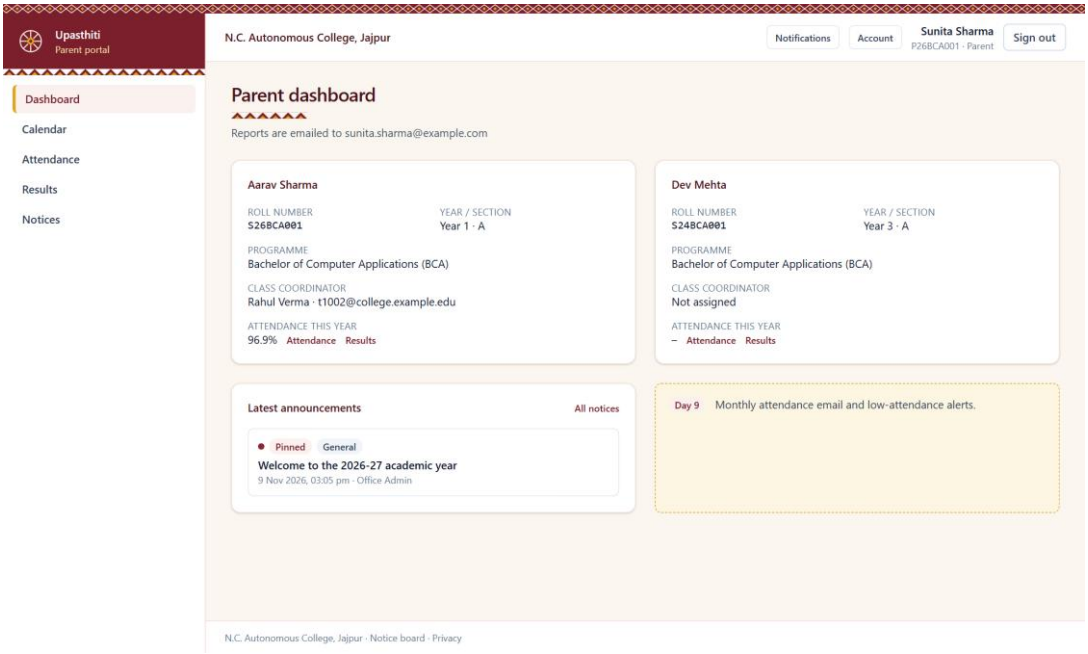


Figure 30: Parent dashboard

6.8 Responsive Design and Mobile Use

Students punch in from their phones, so the screens are designed to work on a small display as well as on a computer:

- The page sets the device-width viewport, so phones show the page at its natural size and do not shrink a desktop layout.
- Layouts use Tailwind's responsive breakpoints. On a phone the side menu is replaced by a compact header with the college mark and a navigation strip, and cards stack in one column.
- Wide tables such as the timetable and the attendance register scroll sideways inside their own container instead of breaking the page.
- The application publishes a web app manifest (standalone display, theme colour), so a student can add it to the phone's home screen like an app.

CHAPTER 7: TESTING

7.1 Testing Strategy

Testing was carried out at three levels throughout development:

- **Static checks:** the TypeScript compiler and ESLint were run after every change to catch type errors and code-quality problems.
- **Unit testing:** individual functions such as the geofence distance calculation, date handling, the attendance percentage, grade calculation, permission checks and CSV/Excel parsing were tested in isolation using the Node.js test runner.
- **End-to-end testing:** automated scripts signed in as each role against a running server with fresh demo data and exercised every feature through real HTTP requests, checking both the allowed actions and that forbidden actions are refused.

7.2 Test Results

Table 9: Automated test suites

Test suite	What it covers	Checks	Result
Unit tests	Distance, dates, attendance %, grades, permissions, audiences, CSV/Excel, rate limits	11	Pass
Sign-in and access	Login, lockout, first-login code, role-based access	24	Pass
Days 2–5	Admin setup, import, geofenced punch, calendar, leave, timetable	121	Pass
Days 6–8	Class attendance, percentage engine, marks and results workflow	103	Pass
Days 9–10	Parent reports, email queue, notices, dashboards, exports, security	69	Pass
Library and notes	Catalogue, issue/return/renew, fines, notes upload, search and access	46	Pass
Total		374	Pass

7.3 Sample Test Cases

Table 10: Sample test cases

ID	Test case	Expected result	Status
TC01	Sign in with a wrong password five times	Account is locked for a period	Pass

ID	Test case	Expected result	Status
TC02	Student punches in from 2 km away	Punch rejected: outside campus	Pass
TC03	Student punches in from inside the campus on a working day	Punch accepted and recorded	Pass
TC04	Student punches from a second, unapproved phone	Punch rejected: device not approved	Pass
TC05	Teacher creates a timetable slot that clashes with another class	Clash reported; slot not saved	Pass
TC06	Teacher edits attendance after 24 hours	Edit request sent to HOD for approval	Pass
TC07	Monthly run finds a student below the critical threshold	Critical alert email queued for the parent	Pass
TC08	Student opens another student's report card URL	Report card is not shown (not found)	Pass
TC09	HOD tries to lock marks by calling the admin-only action	Refused; marks stay Verified	Pass
TC10	Librarian issues a book to a student who already has 3	Refused: limit of 3 books	Pass
TC11	Book returned 6 days late	Fine of ₹12 recorded (₹2 per day)	Pass
TC12	BBA student searches for BCA notes	BCA notes are not shown	Pass
TC13	Upload a file named .pdf whose content is not a PDF	Rejected: file content does not match its type	Pass

7.4 Tools Used for Testing and Debugging

- **Browser developer tools (Chrome DevTools):** to inspect network requests (for example the punch request and its response), console messages and the layout at phone screen sizes.
- **Prisma Studio:** to open the database and check that a punch, a mark or a loan was really stored.
- **Automated scripts:** the end-to-end scripts send real HTTP requests, sign in as each role and print the failing check, which was faster than testing every screen by hand.
- **TypeScript and ESLint:** run after every change (npm run typecheck and npm run lint).

- **Audit log and development outbox:** to confirm that approvals were recorded and that e-mails and one-time codes were generated (in development they are written to an outbox folder instead of being sent).

7.5 Problems Faced and How They Were Solved

Table 11: Problems faced during development

Problem	Cause	Solution
Search by name or title was case-sensitive on PostgreSQL but not on SQLite	SQLite compares text without regard to case, PostgreSQL does not	A single helper (textContains) switches on case-insensitive matching when the production database is used, and all searches call it
Pages that read the signed-in user were rejected by Next.js	Next.js Cache Components requires dynamic data to be read inside a Suspense boundary	Session reads were moved out of layouts and every such page was given a loading screen or a Suspense boundary
Development migrations could not be used on PostgreSQL	The generated SQL is specific to SQLite	Production starts from a fresh PostgreSQL baseline migration; the steps are written in the deployment notes
Slow installs and file-lock errors (EPERM) while developing	The project folder was inside a synchronised cloud folder, which locks node_modules and build files	Those folders are excluded from synchronisation (noted in the README)
Demonstration users were stopped at the privacy-consent page during tests	The consent screen is shown to every user who has not accepted the privacy notice	The seed script records consent for the demonstration accounts so that tests reach the real screens
Prisma settings in package.json were reported as deprecated	Newer Prisma versions read their settings from a config file	The settings were moved to prisma.config.ts

CHAPTER 8: SECURITY AND PRIVACY

- **Password storage:** passwords are hashed with bcrypt and never stored or logged in plain text. A password policy enforces minimum strength.
- **Sessions:** a signed JWT is kept in an HTTP-only, same-site cookie. Changing the password or signing out everywhere invalidates old sessions.
- **Access control:** every page, server action and file download checks the user's role and scope (for example, an HOD sees only their own department).
- **Brute-force protection:** account lockout after 5 failed attempts and rate limits on sign-in and code requests by IP address.
- **Two-step sign-in:** optional one-time code by email for staff accounts.
- **Input validation:** all inputs are validated with zod on the server; database access through Prisma prevents SQL injection.
- **Safe uploads:** file type is checked from the file content, not only the extension; files are stored outside the public folder and served only after a permission check.
- **Security headers:** Content-Security-Policy, X-Frame-Options and related headers are set on every response.
- **Privacy:** explicit consent is collected before any location data is used, with guardian consent for students under 18. Location is read only at the moment of punching, never tracked continuously.
- **Audit trail:** sign-ins, approvals, mark changes, publishing and settings changes are recorded in the audit log.

CHAPTER 9: CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

Upasthiti successfully replaces paper attendance registers and scattered academic records with a single, secure and easy-to-use web application for the BCA and BBA departments of N.C. Autonomous College, Jajpur. The geofenced punch makes proxy attendance difficult, the attendance engine removes calculation errors, and automatic warnings and parent reports make sure that shortages are noticed early. Timetables, marks, notices, library records and study notes are now available to the right people in one place.

Through this project we gained practical experience in full-stack web development with Next.js and React, relational database design with Prisma, authentication and security, working with location data, and automated testing. All planned features were implemented and verified with 374 automated checks.

9.2 Limitations

- GPS accuracy indoors can be poor, so some students may need to step outside to punch.
- Real SMS delivery requires a paid SMS gateway.
- The system currently covers two departments in the demonstration data, though more can be added.

9.3 Future Scope

- Native Android/iOS app with push notifications.
- Face recognition or QR-code check-in in classrooms as an additional verification.
- Online fee payment and hostel management modules.
- Integration with the university examination portal for results.
- Odia and Hindi language versions of the interface.
- Analytics that predict students at risk of shortage early in the semester.

REFERENCES

[1] Next.js Documentation, Vercel. https://nextjs.org/docs

[2] React Documentation, Meta Open Source. https://react.dev

[3] Prisma ORM Documentation, Prisma Data Inc. https://www.prisma.io/docs

[4] TypeScript Handbook, Microsoft. https://www.typescriptlang.org/docs

[5] Tailwind CSS Documentation. https://tailwindcss.com/docs

[6] MDN Web Docs: Geolocation API, Mozilla. https://developer.mozilla.org/docs/Web/API/Geolocation_API

[7] OWASP Cheat Sheet Series: Authentication and Session Management. https://cheatsheetseries.owasp.org

[8] Silberschatz, A., Korth, H. F. and Sudarshan, S., Database System Concepts, McGraw-Hill Education.

[9] Pressman, R. S., Software Engineering: A Practitioner's Approach, McGraw-Hill Education.

APPENDIX: SOURCE CODE AND DEMO ACCESS

A.1 Source Code Repository

The complete source code of the project, including the database schema, migrations, demonstration data and automated tests, is available on GitHub:

https://github.com/bhagyashreepanigrahi76-coder/Upasthiti_Project

A.2 How to Run the Project

```
npm install
copy .env.example .env   (set SESSION_SECRET, CRON_SECRET)
npm run db:migrate
npm run db:seed
npm run dev              (open http://localhost:3000)
```

A.3 Demonstration Accounts

All demonstration accounts use the password Demo@1234.

Table 12: Demonstration accounts

College ID	Role	Opens
ADMIN01	Office admin	Admin portal
T1001	Teacher + HOD, BCA	Portal picker (HOD or Teacher)
T1002	Teacher + Class coordinator, BCA Year 1-A	Teacher portal
T2001	Teacher + HOD, BBA	Portal picker
S26BCA001	BCA student, Year 1	Student portal
S26BBA001	BBA student, Year 1	Student portal
L1001	Librarian	Library portal
P26BCA001	Parent	Parent portal